

Value function Approximation - 2

Master Degree in Control Systems Engineering 2024-2025
University of Padova

Teachers: Prof. Ruggero Carli & Prof. Gian Antonio Susto

Today's lecture

- Short review of last lecture
- Convergence of linear TD(0) method
- TD(λ) algorithm with function approximation : Forward and Backward views

Value Prediction with Function Approximation

- We consider the **prediction problem** of estimating the state-value function v_π from experience generated using policy π .
- The novelty is the use of an *approximate value function parameterized* with parameter vector $\mathbf{w} \in \mathbb{R}^n$

$$\hat{v}(s, \mathbf{w}) \approx v_\pi(s)$$

Interesting class of Function Approximators : linear case

- Represent state by a *feature vector*

$$\mathbf{x}(S) = \begin{pmatrix} \mathbf{x}_1(S) \\ \vdots \\ \mathbf{x}_n(S) \end{pmatrix}$$

- Represent value function by a linear combination of features

$$\hat{v}(S, \mathbf{w}) = \mathbf{x}(S)^\top \mathbf{w} = \sum_{j=1}^n \mathbf{x}_j(S) \mathbf{w}_j$$

Value Prediction with Function Approximation

What performance measures are appropriate for evaluating function approximation methods?

MEAN – SQUARED ERROR (MSE)

$$\overline{\text{VE}}(\mathbf{w}) \doteq \sum_{s \in \mathcal{S}} \mu(s) \left[v_{\pi}(s) - \hat{v}(s, \mathbf{w}) \right]^2$$

μ is a distribution

Value Prediction with Function Approximation

What performance measures are appropriate for evaluating function approximation methods?

MEAN – SQUARED ERROR (MSE)

$$\overline{\text{VE}}(\mathbf{w}) \doteq \sum_{s \in \mathcal{S}} \mu(s) \left[v_{\pi}(s) - \hat{v}(s, \mathbf{w}) \right]^2 \quad \rightarrow \quad J(\mathbf{w}) = \mathbb{E}_{\pi} \left[(v_{\pi}(S) - \hat{v}(S, \mathbf{w}))^2 \right]$$

μ is a distribution

In continuing tasks $\mu(s)$ is the **stationary distribution** under π

Value Prediction with Function Approximation

What performance measures are appropriate for evaluating function approximation methods?

MEAN – SQUARED ERROR (MSE)

$$\overline{\text{VE}}(\mathbf{w}) \doteq \sum_{s \in \mathcal{S}} \mu(s) \left[v_{\pi}(s) - \hat{v}(s, \mathbf{w}) \right]^2 \quad \rightarrow \quad J(\mathbf{w}) = \mathbb{E}_{\pi} \left[(v_{\pi}(S) - \hat{v}(S, \mathbf{w}))^2 \right]$$

μ is a distribution

In continuing tasks $\mu(s)$ is the **stationary distribution** under π

Ideal goal : finding $\mathbf{w}^*$ such that $\overline{\text{VE}}(\mathbf{w}^*) \leq \overline{\text{VE}}(\mathbf{w})$ for all $\mathbf{w}$
(global optimal)

Value Prediction with Function Approximation

What performance measures are appropriate for evaluating function approximation methods?

MEAN – SQUARED ERROR (MSE)

$$\overline{\text{VE}}(\mathbf{w}) \doteq \sum_{s \in \mathcal{S}} \mu(s) \left[v_{\pi}(s) - \hat{v}(s, \mathbf{w}) \right]^2 \quad \rightarrow \quad J(\mathbf{w}) = \mathbb{E}_{\pi} \left[(v_{\pi}(S) - \hat{v}(S, \mathbf{w}))^2 \right]$$

μ is a distribution

In continuing tasks $\mu(s)$ is the **stationary distribution** under π

Ideal goal : finding $\mathbf{w}^*$ such that $\overline{\text{VE}}(\mathbf{w}^*) \leq \overline{\text{VE}}(\mathbf{w})$ for all $\mathbf{w}$

(global optimal)

 sometimes possible for simple function approximators (e.g., *linear ones*)

Value Prediction with Function Approximation

What performance measures are appropriate for evaluating function approximation methods?

MEAN – SQUARED ERROR (MSE)

$$\overline{\text{VE}}(\mathbf{w}) \doteq \sum_{s \in \mathcal{S}} \mu(s) \left[v_{\pi}(s) - \hat{v}(s, \mathbf{w}) \right]^2 \quad \rightarrow \quad J(\mathbf{w}) = \mathbb{E}_{\pi} \left[(v_{\pi}(S) - \hat{v}(S, \mathbf{w}))^2 \right]$$

μ is a distribution

In continuing tasks $\mu(s)$ is the **stationary distribution** under π

Ideal goal : finding $\mathbf{w}^*$ such that $\overline{\text{VE}}(\mathbf{w}^*) \leq \overline{\text{VE}}(\mathbf{w})$ for all $\mathbf{w}$

(global optimal)

- 
- sometimes possible for simple function approximators (e.g., *linear ones*)
 - rarely possible for complex function approximators

Value Prediction with Function Approximation

What performance measures are appropriate for evaluating function approximation methods?

MEAN – SQUARED ERROR (MSE)

$$\overline{\text{VE}}(\mathbf{w}) \doteq \sum_{s \in \mathcal{S}} \mu(s) \left[v_{\pi}(s) - \hat{v}(s, \mathbf{w}) \right]^2 \quad \rightarrow \quad J(\mathbf{w}) = \mathbb{E}_{\pi} \left[(v_{\pi}(S) - \hat{v}(S, \mathbf{w}))^2 \right]$$

μ is a distribution

In continuing tasks $\mu(s)$ is the **stationary distribution** under π

Ideal goal : finding $\mathbf{w}^*$ such that $\overline{\text{VE}}(\mathbf{w}^*) \leq \overline{\text{VE}}(\mathbf{w})$ for all $\mathbf{w}$
(global optimal)

Realistic goal (for complex approximators): to converge to a vector $\mathbf{w}^*$ such that

$$\overline{\text{VE}}(\mathbf{w}^*) \leq \overline{\text{VE}}(\mathbf{w}) \quad \text{for all } \mathbf{w} \text{ in a neighborhood of } \mathbf{w}^*$$

(local optimum)

Value Prediction with Function Approximation

What performance measures are appropriate for evaluating function approximation methods?

MEAN – SQUARED ERROR (MSE)

$$\overline{\text{VE}}(\mathbf{w}) \doteq \sum_{s \in \mathcal{S}} \mu(s) \left[v_{\pi}(s) - \hat{v}(s, \mathbf{w}) \right]^2 \quad \rightarrow \quad J(\mathbf{w}) = \mathbb{E}_{\pi} \left[(v_{\pi}(S) - \hat{v}(S, \mathbf{w}))^2 \right]$$

μ is a distribution

In continuing tasks $\mu(s)$ is the **stationary distribution** under π

In particular, *in the linear case there is only one optimum* (or, in degenerate cases, one set of equally good optima)

Value function approximation by stochastic gradient descent

Goal : find parameter vector $\mathbf{w}$ minimising mean-squared error between approximate value fn $\hat{v}(s, \mathbf{w})$ and true value fn $v_\pi(s)$

$$J(\mathbf{w}) = \mathbb{E}_\pi \left[(v_\pi(s) - \hat{v}(s, \mathbf{w}))^2 \right]$$

- *Gradient descent* finds a local minimum

$$\begin{aligned} \Delta \mathbf{w} &= -\frac{1}{2} \alpha \nabla_{\mathbf{w}} J(\mathbf{w}) \\ &= \alpha \mathbb{E}_\pi [(v_\pi(s) - \hat{v}(s, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(s, \mathbf{w})] \end{aligned}$$

- *Stochastic gradient descent* *samples* the gradient

$$\Delta \mathbf{w} = \alpha (v_\pi(s) - \hat{v}(s, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(s, \mathbf{w})$$

Value function approximation by stochastic gradient descent

Goal : find parameter vector $\mathbf{w}$ minimising mean-squared error between approximate value fn $\hat{v}(s, \mathbf{w})$ and true value fn $v_\pi(s)$

$$J(\mathbf{w}) = \mathbb{E}_\pi \left[(v_\pi(s) - \hat{v}(s, \mathbf{w}))^2 \right]$$

- *Gradient descent* finds a local minimum

$$\begin{aligned} \Delta \mathbf{w} &= -\frac{1}{2} \alpha \nabla_{\mathbf{w}} J(\mathbf{w}) \\ &= \alpha \mathbb{E}_\pi [(v_\pi(s) - \hat{v}(s, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(s, \mathbf{w})] \end{aligned}$$

- *Stochastic gradient descent* *samples* the gradient

$$\Delta \mathbf{w} = \alpha (v_\pi(s) - \hat{v}(s, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(s, \mathbf{w})$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \Delta \mathbf{w}$$

Value function approximation by stochastic gradient descent

Goal : find parameter vector $\mathbf{w}$ minimising mean-squared error between approximate value fn $\hat{v}(s, \mathbf{w})$ and true value fn $v_\pi(s)$

$$J(\mathbf{w}) = \mathbb{E}_\pi \left[(v_\pi(s) - \hat{v}(s, \mathbf{w}))^2 \right]$$

- *Gradient descent* finds a local minimum

$$\begin{aligned} \Delta \mathbf{w} &= -\frac{1}{2} \alpha \nabla_{\mathbf{w}} J(\mathbf{w}) \\ &= \alpha \mathbb{E}_\pi [(v_\pi(s) - \hat{v}(s, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(s, \mathbf{w})] \end{aligned}$$

- *Stochastic gradient descent* *samples* the gradient

$$\Delta \mathbf{w} = \alpha (v_\pi(s) - \hat{v}(s, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(s, \mathbf{w})$$

it is given by an oracle

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \Delta \mathbf{w}$$

Value function approximation by stochastic gradient descent

Stochastic gradient-descent

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha \left[v_\pi(S_t) - \hat{v}(S_t, \mathbf{w}_t) \right] \nabla \hat{v}(S_t, \mathbf{w}_t)$$

The convergence results for SGD methods assume that α decreases over time

$$\sum_{t=1}^{\infty} \alpha_t = \infty \quad \sum_{t=1}^{\infty} \alpha_t^2 < \infty \quad \left(\alpha_t = \frac{1}{t} \right)$$

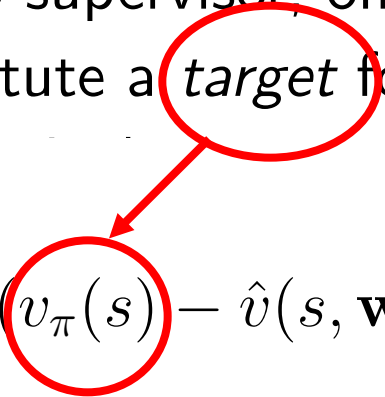
Convergence to a local optimum

Incremental Prediction Algorithms

- Have assumed true value function $v_{\pi}(s)$ given by supervisor
- But in RL there is no supervisor, only rewards
- In practice, we substitute a *target* for $v_{\pi}(s)$

Incremental Prediction Algorithms

- Have assumed true value function $v_\pi(s)$ given by supervisor
- But in RL there is no supervisor, only rewards
- In practice, we substitute a *target* for $v_\pi(s)$


$$\Delta \mathbf{w} = \alpha (v_\pi(s) - \hat{v}(s, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(s, \mathbf{w})$$

Incremental Prediction Algorithms

- Have assumed true value function $v_\pi(s)$ given by supervisor
- But in RL there is no supervisor, only rewards
- In practice, we substitute a *target* for $v_\pi(s)$

- For MC, the target is the return G_t

$$\Delta \mathbf{w} = \alpha(\textcolor{red}{G}_t - \hat{v}(S_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w})$$

- For TD(0), the target is the TD target $R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w})$

$$\Delta \mathbf{w} = \alpha(\textcolor{red}{R}_{t+1} + \gamma \hat{v}(\textcolor{red}{S}_{t+1}, \mathbf{w}) - \hat{v}(S_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w})$$

- For TD(λ), the target is the λ -return G_t^λ

$$\Delta \mathbf{w} = \alpha(\textcolor{red}{G}_t^\lambda - \hat{v}(S_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w})$$

Monte – Carlo with Value Function Approximation

- Return G_t is an unbiased, noisy sample of true value $v_\pi(S_t)$
- Can therefore apply supervised learning to “training data”:

$$\langle S_1, G_1 \rangle, \langle S_2, G_2 \rangle, \dots, \langle S_T, G_T \rangle$$

- For example, using *linear Monte-Carlo policy evaluation*

$$\begin{aligned}\Delta \mathbf{w} &= \alpha(\textcolor{red}{G}_t - \hat{v}(S_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}) \\ &= \alpha(G_t - \hat{v}(S_t, \mathbf{w})) \mathbf{x}(S_t) \quad \textcolor{red}{(linear case)}\end{aligned}$$

Monte – Carlo with Value Function Approximation

Gradient Monte Carlo Algorithm for Estimating $\hat{v} \approx v_\pi$

Input: $\pi, \hat{v} : \mathcal{S} \times \mathbb{R}^d \rightarrow \mathbb{R}$ ($\hat{v}(\text{terminal}, \cdot) = 0$), step-size α

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g. $\mathbf{w} = 0$)

Repeat (for each episode) :

Generate an episode $S_0, A_0, R_1, S_1, A_1, \dots, R_T, S_T$ using π

Repeat (for each step of the episode, $t = 0, 1, \dots, T - 1$) :

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [G_t - \hat{v}(S_t, \mathbf{w})] \nabla \hat{v}(S_t, \mathbf{w})$$

Monte – Carlo with Value Function Approximation

Gradient Monte Carlo Algorithm for Estimating $\hat{v} \approx v_\pi$

Input: π , $\hat{v} : \mathcal{S} \times \mathbb{R}^d \rightarrow \mathbb{R}$ ($\hat{v}(\text{terminal}, \cdot) = 0$), step-size α

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g. $\mathbf{w} = 0$)

Repeat (for each episode) :

Generate an episode $S_0, A_0, R_1, S_1, A_1, \dots, R_T, S_T$ using π

Repeat (for each step of the episode, $t = 0, 1, \dots, T - 1$) :

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [G_t - \hat{v}(S_t, \mathbf{w})] \nabla \hat{v}(S_t, \mathbf{w})$$

Remember that $\mathbb{E}[G_t | S_t = s] = v_\pi(S_t)$

$\Rightarrow$ MC target G_t is by definition an unbiased estimate of $v_\pi(S_t)$

$\Rightarrow$ SGD converges to a locally optimal approximation of $v_\pi(S_t)$ assuming

$$\sum_{t=1}^{\infty} \alpha_t = \infty \quad \sum_{t=1}^{\infty} \alpha_t^2 < \infty$$

Monte – Carlo with Value Function Approximation

Gradient Monte Carlo Algorithm for Estimating $\hat{v} \approx v_\pi$

Input: π , $\hat{v} : \mathcal{S} \times \mathbb{R}^d \rightarrow \mathbb{R}$ ($\hat{v}(\text{terminal}, \cdot) = 0$), step-size α

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g. $\mathbf{w} = 0$)

Repeat (for each episode) :

 Generate an episode $S_0, A_0, R_1, S_1, A_1, \dots, R_T, S_T$ using π

 Repeat (for each step of the episode, $t = 0, 1, \dots, T - 1$) :

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [G_t - \hat{v}(S_t, \mathbf{w})] \nabla \hat{v}(S_t, \mathbf{w})$$

Monte Carlo evaluation converges to *a local optimum* in the general case of nonlinear function approximation

When adopting linear function approximation, Monte Carlo evaluation converges to the *global optimum*

TD(0) Learning with Value Function Approximation

- The TD-target $R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w})$ is a *biased* sample of true value $v_\pi(S_t)$
- Can still apply supervised learning to “training data”:

$$\langle S_1, R_2 + \gamma \hat{v}(S_2, \mathbf{w}) \rangle, \langle S_2, R_3 + \gamma \hat{v}(S_3, \mathbf{w}) \rangle, \dots, \langle S_{T-1}, R_T \rangle$$

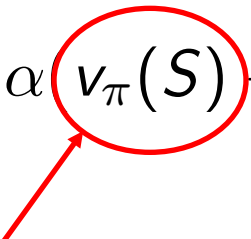
- For example, using *linear TD(0)*

$$\begin{aligned} \Delta \mathbf{w} &= \alpha (\textcolor{red}{R} + \gamma \hat{v}(\textcolor{red}{S}', \mathbf{w}) - \hat{v}(S, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S, \mathbf{w}) \\ &= \alpha \delta \mathbf{x}(S) \quad (\textcolor{red}{linear case}) \end{aligned}$$

- Linear TD(0) converges (close) to global optimum

TD(0) Learning with Value Function Approximation

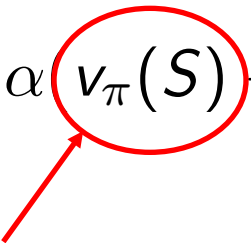
An important observation:

$$\Delta \mathbf{w} = \alpha (v_{\pi}(S) - \hat{v}(S, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S, \mathbf{w})$$


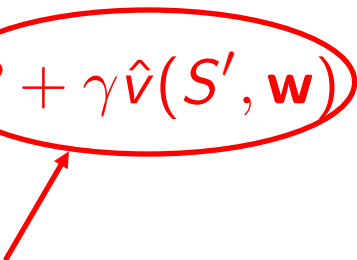
Target does not depend on w

TD(0) Learning with Value Function Approximation

An important observation:

$$\Delta \mathbf{w} = \alpha (v_{\pi}(S) - \hat{v}(S, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S, \mathbf{w})$$


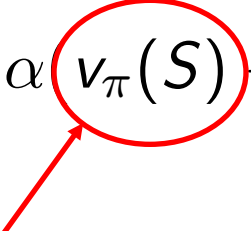
Target does not depend on w

$$\Delta \mathbf{w} = \alpha (R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S, \mathbf{w})$$


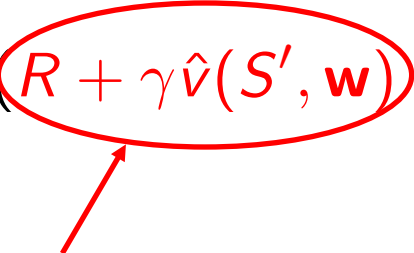
It depends on w (a typical property of all bootstrapping methods)

TD(0) Learning with Value Function Approximation

An important observation:

$$\Delta \mathbf{w} = \alpha (v_{\pi}(S) - \hat{v}(S, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S, \mathbf{w})$$


Target does not depend on w

$$\Delta \mathbf{w} = \alpha (R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S, \mathbf{w})$$


It depends on w (a typical property of all bootstrapping methods)

They are not true gradient descent method: *they take into account the effect of changing the weight vector w_t on the estimate but ignore its effect on the target*

They are **semi-gradient methods**

TD(0) Learning with Value Function Approximation

Semi-gradient TD(0) for estimating $\hat{v} \approx v_\pi$

Input: π , $\hat{v} : \mathcal{S} \times \mathbb{R}^d \rightarrow \mathbb{R}$ ($\hat{v}(\text{terminal}, \cdot) = 0$), step-size α

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g. $\mathbf{w} = 0$)

Repeat (for each episode) :

 Initialize S

 Repeat (for each step of the episode) :

 Choose $A \sim \pi(\cdot | S)$

 Take action A , observe R, S'

$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})] \nabla \hat{v}(S, \mathbf{w})$

$S \leftarrow S'$

 Until S' is terminal

TD(0) Learning with Value Function Approximation

Semi-gradient TD(0) for estimating $\hat{v} \approx v_\pi$

Input: π , $\hat{v} : \mathcal{S} \times \mathbb{R}^d \rightarrow \mathbb{R}$ ($\hat{v}(\text{terminal}, \cdot) = 0$), step-size α

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g. $\mathbf{w} = 0$)

Repeat (for each episode) :

 Initialize S

 Repeat (for each step of the episode) :

 Choose $A \sim \pi(\cdot | S)$

 Take action A , observe R, S'

$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})] \nabla \hat{v}(S, \mathbf{w})$

$S \leftarrow S'$

 Until S' is terminal

- Since bootstrapping targets are biased, they do not have the general convergence properties of the MC targets
- They do converge reliably in important cases as the linear case (next slides)

TD(0) Learning with Value Function Approximation- Linear case

The semi-gradient TD(0) algorithm converges under linear function approximation, but this does not follow from general results on SGD; a separate theorem is necessary.

TD(0) Learning with Value Function Approximation- Linear case

The semi-gradient TD(0) algorithm converges under linear function approximation, but this does not follow from general results on SGD; a separate theorem is necessary.

In the following we provide analysis for **continuing case**

TD(0) Learning with Value Function Approximation- Linear case

The semi-gradient TD(0) algorithm converges under linear function approximation, but this does not follow from general results on SGD; a separate theorem is necessary.

In the following we provide analysis for **continuing case**

The update at each time t is

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})] \nabla \hat{v}(S, \mathbf{w})$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha (R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t) - \hat{v}(S_t, \mathbf{w}_t)) \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}_t)$$

TD(0) Learning with Value Function Approximation- Linear case

The semi-gradient TD(0) algorithm converges under linear function approximation, but this does not follow from general results on SGD; a separate theorem is necessary.

In the following we provide analysis for **continuing case**

The update at each time t is

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})] \nabla \hat{v}(S, \mathbf{w})$$

$$\begin{aligned} \mathbf{w}_{t+1} &= \mathbf{w}_t + \alpha (R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t) - \hat{v}(S_t, \mathbf{w}_t)) \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}_t) \\ &\doteq \mathbf{w}_t + \alpha (R_{t+1} + \gamma \mathbf{w}_t^\top \mathbf{x}_{t+1} - \mathbf{w}_t^\top \mathbf{x}_t) \mathbf{x}_t \end{aligned}$$

Remember

$$\hat{v}(S, \mathbf{w}) = \mathbf{x}(S)^\top \mathbf{w}$$

Shorthand

$$\mathbf{x}_t = \mathbf{x}(S_t)$$

TD(0) Learning with Value Function Approximation- Linear case

The semi-gradient TD(0) algorithm converges under linear function approximation, but this does not follow from general results on SGD; a separate theorem is necessary.

In the following we provide analysis for **continuing case**

The update at each time t is

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})] \nabla \hat{v}(S, \mathbf{w})$$

$$\begin{aligned} \mathbf{w}_{t+1} &= \mathbf{w}_t + \alpha (R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t) - \hat{v}(S_t, \mathbf{w}_t)) \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}_t) \\ &\doteq \mathbf{w}_t + \alpha \left(R_{t+1} + \gamma \mathbf{w}_t^\top \mathbf{x}_{t+1} - \mathbf{w}_t^\top \mathbf{x}_t \right) \mathbf{x}_t \\ &= \mathbf{w}_t + \alpha \left(R_{t+1} \mathbf{x}_t - \mathbf{x}_t (\mathbf{x}_t - \gamma \mathbf{x}_{t+1})^\top \mathbf{w}_t \right) \end{aligned}$$

TD(0) Learning with Value Function Approximation- Linear case

Suppose the system has reached steady state

⇒ the expected next weight vector $\mathbb{E}[\mathbf{w}_{t+1}|\mathbf{w}_t]$ evolves as

$$\mathbb{E}[\mathbf{w}_{t+1}|\mathbf{w}_t] = \mathbb{E}\left[\left(\mathbf{w}_t + \alpha(R_{t+1}\mathbf{x}_t - \mathbf{x}_t(\mathbf{x}_t - \gamma\mathbf{x}_{t+1})^T\mathbf{w}_t)\right) \mid \mathbf{w}_t\right]$$

Remember

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha(R_{t+1}\mathbf{x}_t - \mathbf{x}_t(\mathbf{x}_t - \gamma\mathbf{x}_{t+1})^T\mathbf{w}_t)$$

TD(0) Learning with Value Function Approximation- Linear case

Suppose the system has reached steady state

⇒ the expected next weight vector $\mathbb{E}[\mathbf{w}_{t+1}|\mathbf{w}_t]$ evolves as

$$\begin{aligned}\mathbb{E}[\mathbf{w}_{t+1}|\mathbf{w}_t] &= \mathbb{E}\left[\left(\mathbf{w}_t + \alpha(R_{t+1}\mathbf{x}_t - \mathbf{x}_t(\mathbf{x}_t - \gamma\mathbf{x}_{t+1})^T\mathbf{w}_t)\right) \mid \mathbf{w}_t\right] \\ &= \mathbf{w}_t + \alpha\mathbb{E}[R_{t+1}\mathbf{x}_t] - \alpha\mathbb{E}\left[\mathbf{x}_t(\mathbf{x}_t - \gamma\mathbf{x}_{t+1})^T\right]\mathbf{w}_t\end{aligned}$$

Notice

$$\mathbb{E}[\mathbf{w}_t|\mathbf{w}_t] = \mathbf{w}_t$$

$$\mathbb{E}\left[\mathbf{x}_t(\mathbf{x}_t - \gamma\mathbf{x}_{t+1})^T\mathbf{w}_t \mid \mathbf{w}_t\right] = \mathbb{E}\left[\mathbf{x}_t(\mathbf{x}_t - \gamma\mathbf{x}_{t+1})^T\right]\mathbf{w}_t$$

TD(0) Learning with Value Function Approximation- Linear case

Suppose the system has reached steady state

$\Rightarrow$ the expected next weight vector $\mathbb{E}[\mathbf{w}_{t+1}|\mathbf{w}_t]$ evolves as

$$\begin{aligned}\mathbb{E}[\mathbf{w}_{t+1}|\mathbf{w}_t] &= \mathbb{E}\left[\left(\mathbf{w}_t + \alpha(R_{t+1}\mathbf{x}_t - \mathbf{x}_t(\mathbf{x}_t - \gamma\mathbf{x}_{t+1})^T\mathbf{w}_t)\right) \mid \mathbf{w}_t\right] \\ &= \mathbf{w}_t + \alpha\mathbb{E}[R_{t+1}\mathbf{x}_t] - \alpha\mathbb{E}\left[\mathbf{x}_t(\mathbf{x}_t - \gamma\mathbf{x}_{t+1})^T\right]\mathbf{w}_t \\ &= \mathbf{w}_t + \alpha(\mathbf{b} - \mathbf{A}\mathbf{w}_t)\end{aligned}$$

where $\mathbf{b} \doteq \mathbb{E}[R_{t+1}\mathbf{x}_t] \in \mathbb{R}^d$ and $\mathbf{A} \doteq \mathbb{E}\left[\mathbf{x}_t(\mathbf{x}_t - \gamma\mathbf{x}_{t+1})^T\right] \in \mathbb{R}^d \times \mathbb{R}^d$

TD(0) Learning with Value Function Approximation- Linear case

Suppose the system has reached steady state

$\Rightarrow$ the expected next weight vector $\mathbb{E}[\mathbf{w}_{t+1}|\mathbf{w}_t]$ evolves as

$$\begin{aligned}\mathbb{E}[\mathbf{w}_{t+1}|\mathbf{w}_t] &= \mathbb{E}\left[\left(\mathbf{w}_t + \alpha(R_{t+1}\mathbf{x}_t - \mathbf{x}_t(\mathbf{x}_t - \gamma\mathbf{x}_{t+1})^T\mathbf{w}_t)\right) \mid \mathbf{w}_t\right] \\ &= \mathbf{w}_t + \alpha\mathbb{E}[R_{t+1}\mathbf{x}_t] - \alpha\mathbb{E}\left[\mathbf{x}_t(\mathbf{x}_t - \gamma\mathbf{x}_{t+1})^T\right]\mathbf{w}_t \\ &= \mathbf{w}_t + \alpha(\mathbf{b} - \mathbf{A}\mathbf{w}_t)\end{aligned}$$

where $\mathbf{b} \doteq \mathbb{E}[R_{t+1}\mathbf{x}_t] \in \mathbb{R}^d$ and $\mathbf{A} \doteq \mathbb{E}\left[\mathbf{x}_t(\mathbf{x}_t - \gamma\mathbf{x}_{t+1})^T\right] \in \mathbb{R}^d \times \mathbb{R}^d$

If the system converges it must converge to the weight vector $\mathbf{w}_{\text{TD}}$ s.t.

$$\mathbf{b} - \mathbf{A}\mathbf{w}_{\text{TD}} = \mathbf{0}$$

TD(0) Learning with Value Function Approximation- Linear case

Suppose the system has reached steady state

$\Rightarrow$ the expected next weight vector $\mathbb{E}[\mathbf{w}_{t+1}|\mathbf{w}_t]$ evolves as

$$\begin{aligned}\mathbb{E}[\mathbf{w}_{t+1}|\mathbf{w}_t] &= \mathbb{E}\left[\left(\mathbf{w}_t + \alpha(R_{t+1}\mathbf{x}_t - \mathbf{x}_t(\mathbf{x}_t - \gamma\mathbf{x}_{t+1})^T\mathbf{w}_t)\right) \mid \mathbf{w}_t\right] \\ &= \mathbf{w}_t + \alpha\mathbb{E}[R_{t+1}\mathbf{x}_t] - \alpha\mathbb{E}\left[\mathbf{x}_t(\mathbf{x}_t - \gamma\mathbf{x}_{t+1})^T\right]\mathbf{w}_t \\ &= \mathbf{w}_t + \alpha(\mathbf{b} - \mathbf{A}\mathbf{w}_t)\end{aligned}$$

where $\mathbf{b} \doteq \mathbb{E}[R_{t+1}\mathbf{x}_t] \in \mathbb{R}^d$ and $\mathbf{A} \doteq \mathbb{E}\left[\mathbf{x}_t(\mathbf{x}_t - \gamma\mathbf{x}_{t+1})^T\right] \in \mathbb{R}^d \times \mathbb{R}^d$

If the system converges it must converge to the weight vector $\mathbf{w}_{\text{TD}}$ s.t.

$$\mathbf{b} - \mathbf{A}\mathbf{w}_{\text{TD}} = \mathbf{0}$$

$$\Rightarrow \mathbf{b} = \mathbf{A}\mathbf{w}_{\text{TD}}$$

$$\Rightarrow \mathbf{w}_{\text{TD}} \doteq \mathbf{A}^{-1}\mathbf{b}.$$

TD(0) Learning with Value Function Approximation- Linear case

$$\mathbb{E}[\mathbf{w}_{t+1}|\mathbf{w}_t] = \mathbf{w}_t + \alpha(\mathbf{b} - \mathbf{A}\mathbf{w}_t)$$



$$\mathbb{E}[\mathbf{w}_{t+1}|\mathbf{w}_t] = (\mathbf{I} - \alpha\mathbf{A})\mathbf{w}_t + \alpha\mathbf{b}$$

TD(0) Learning with Value Function Approximation- Linear case

$$\mathbb{E}[\mathbf{w}_{t+1}|\mathbf{w}_t] = \mathbf{w}_t + \alpha(\mathbf{b} - \mathbf{A}\mathbf{w}_t)$$



$$\mathbb{E}[\mathbf{w}_{t+1}|\mathbf{w}_t] = (\mathbf{I} - \alpha\mathbf{A})\mathbf{w}_t + \alpha\mathbf{b} \xrightarrow{\quad ? \quad} \mathbf{w}_{\text{TD}}$$

TD(0) Learning with Value Function Approximation- Linear case

$$\mathbb{E}[\mathbf{w}_{t+1}|\mathbf{w}_t] = \mathbf{w}_t + \alpha(\mathbf{b} - \mathbf{A}\mathbf{w}_t)$$



$$\mathbb{E}[\mathbf{w}_{t+1}|\mathbf{w}_t] = (\mathbf{I} - \alpha\mathbf{A})\mathbf{w}_t + \alpha\mathbf{b} \xrightarrow{\quad ? \quad} \mathbf{w}_{\text{TD}}$$

- Only matrix $\mathbf{A}$ is important to convergence!!!

TD(0) Learning with Value Function Approximation- Linear case

$$\mathbb{E}[\mathbf{w}_{t+1}|\mathbf{w}_t] = \mathbf{w}_t + \alpha(\mathbf{b} - \mathbf{A}\mathbf{w}_t)$$



$$\mathbb{E}[\mathbf{w}_{t+1}|\mathbf{w}_t] = (\mathbf{I} - \alpha\mathbf{A})\mathbf{w}_t + \alpha\mathbf{b} \xrightarrow{\quad ? \quad} \mathbf{w}_{\text{TD}}$$

- Only matrix $\mathbf{A}$ is important to convergence!!!
- In the continuing case with $\gamma < 1$ it is possible to see that there exists $\bar{\alpha}$ such that for $0 < \alpha \leq \bar{\alpha}$ then the matrix $\mathbf{I} - \alpha\mathbf{A}$ is stable
(more details in the book at pages 166-167)

TD(0) Learning with Value Function Approximation- Linear case

$$\mathbb{E}[\mathbf{w}_{t+1}|\mathbf{w}_t] = \mathbf{w}_t + \alpha(\mathbf{b} - \mathbf{A}\mathbf{w}_t)$$



$$\mathbb{E}[\mathbf{w}_{t+1}|\mathbf{w}_t] = (\mathbf{I} - \alpha\mathbf{A})\mathbf{w}_t + \alpha\mathbf{b} \xrightarrow{\quad ? \quad} \mathbf{w}_{\text{TD}}$$

- Only matrix $\mathbf{A}$ is important to convergence!!!
- In the continuing case with $\gamma < 1$ it is possible to see that there exists $\bar{\alpha}$ such that for $0 < \alpha \leq \bar{\alpha}$ then the matrix $\mathbf{I} - \alpha\mathbf{A}$ is stable
- Bound on the distance from the optimal solution (it might be large if $\gamma \rightarrow 1$)

$$\overline{\text{VE}}(\mathbf{w}_{\text{TD}}) \leq \frac{1}{1 - \gamma} \min_{\mathbf{w}} \overline{\text{VE}}(\mathbf{w})$$

TD(0) Learning with Value Function Approximation- Linear case

$$\mathbf{A} \doteq \mathbb{E} \left[\mathbf{x}_t (\mathbf{x}_t - \gamma \mathbf{x}_{t+1})^\top \right] \in \mathbb{R}^d \times \mathbb{R}^d$$

$$\mathbf{A} = \sum_s \mu(s) \sum_a \pi(a|s) \sum_{r,s'} p(r, s' | s, a) \mathbf{x}(s) (\mathbf{x}(s) - \gamma \mathbf{x}(s'))^\top$$

μ stationary distribution under π

TD(0) Learning with Value Function Approximation- Linear case

$$\mathbf{A} \doteq \mathbb{E} \left[\mathbf{x}_t (\mathbf{x}_t - \gamma \mathbf{x}_{t+1})^\top \right] \in \mathbb{R}^d \times \mathbb{R}^d$$

$$\begin{aligned} \mathbf{A} &= \sum_s \mu(s) \sum_a \pi(a|s) \sum_{r,s'} p(r, s' | s, a) \mathbf{x}(s) (\mathbf{x}(s) - \gamma \mathbf{x}(s'))^\top \\ &= \sum_s \mu(s) \sum_{s'} p(s' | s) \mathbf{x}(s) (\mathbf{x}(s) - \gamma \mathbf{x}(s'))^\top \end{aligned}$$

μ stationary distribution under π

$p(s'|s)$ is the transition probability from s to s'

TD(0) Learning with Value Function Approximation- Linear case

$$\mathbf{A} \doteq \mathbb{E} \left[\mathbf{x}_t (\mathbf{x}_t - \gamma \mathbf{x}_{t+1})^\top \right] \in \mathbb{R}^d \times \mathbb{R}^d$$

$$\begin{aligned} \mathbf{A} &= \sum_s \mu(s) \sum_a \pi(a|s) \sum_{r,s'} p(r, s' | s, a) \mathbf{x}(s) (\mathbf{x}(s) - \gamma \mathbf{x}(s'))^\top \\ &= \sum_s \mu(s) \sum_{s'} p(s' | s) \mathbf{x}(s) (\mathbf{x}(s) - \gamma \mathbf{x}(s'))^\top \\ &= \sum_s \mu(s) \mathbf{x}(s) \left(\mathbf{x}(s) - \gamma \sum_{s'} p(s' | s) \mathbf{x}(s') \right)^\top \end{aligned}$$

μ stationary distribution under π

$p(s'|s)$ is the transition probability from s to s'

TD(0) Learning with Value Function Approximation- Linear case

$$\mathbf{A} \doteq \mathbb{E} \left[\mathbf{x}_t (\mathbf{x}_t - \gamma \mathbf{x}_{t+1})^\top \right] \in \mathbb{R}^d \times \mathbb{R}^d$$

$$\begin{aligned} \mathbf{A} &= \sum_s \mu(s) \sum_a \pi(a|s) \sum_{r,s'} p(r, s' | s, a) \mathbf{x}(s) (\mathbf{x}(s) - \gamma \mathbf{x}(s'))^\top \\ &= \sum_s \mu(s) \sum_{s'} p(s' | s) \mathbf{x}(s) (\mathbf{x}(s) - \gamma \mathbf{x}(s'))^\top \\ &= \sum_s \mu(s) \mathbf{x}(s) \left(\mathbf{x}(s) - \gamma \sum_{s'} p(s' | s) \mathbf{x}(s') \right)^\top \\ &= \mathbf{X}^\top \mathbf{D} (\mathbf{I} - \gamma \mathbf{P}) \mathbf{X}, \end{aligned}$$

μ stationary distribution under π

$p(s'|s)$ is the transition probability from s to s'

P is the matrix of this probabilities ($|S| \times |S|$)

D is a diagonal matrix with $\mu(s)$ on its diagonal ($|S| \times |S|$)

X is the matrix with $\mathbf{x}(s)$ on its rows ($|S| \times d$)

Derivation matrix A is not required for the final exam

n-step Learning with Value Function Approximation

Extension to n-step Learning

n-step return $G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$

n-step Learning with Value Function Approximation

Extension to n-step Learning

n-step return $G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$



$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n \hat{v}(S_{t+n}, \mathbf{w}_t)$$

n-step Learning with Value Function Approximation

Extension to n-step Learning

n-step return $G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n V(S_{t+n})$



$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{n-1} R_{t+n} + \gamma^n \hat{v}(S_{t+n}, \mathbf{w}_t)$$

n-step Learning with Value Function Approximation

n-step semi-gradient TD for estimating $\hat{v} \approx v_\pi$

Input: $\pi, \hat{v} : \mathcal{S} \times \mathbb{R}^d \rightarrow \mathbb{R}$ ($\hat{v}(\text{terminal}, \cdot) = 0$), step-size α , integer n

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g. $\mathbf{w} = 0$)

Repeat (for each episode) :

Generate an episode $S_0, A_0, R_1, S_1, A_1, \dots, R_T, S_T$ using π

Repeat (for each step of the episode, $t = 0, 1, \dots, T - 1$) :

$$G \leftarrow \sum_{i=t+1}^{\min(t+n, T)} \gamma^{(i-t-1)} R_i$$

$$\text{If } t+n < T, \text{ then : } G \leftarrow G + \gamma^n \hat{v}(S_{t+n}, \mathbf{w})$$

$$\mathbf{w} \leftarrow \mathbf{w} + \alpha [G - \hat{v}(S_t, \mathbf{w})] \nabla \hat{v}(S_t, \mathbf{w})$$

TD(λ) with Value Function Approx. : Forward view

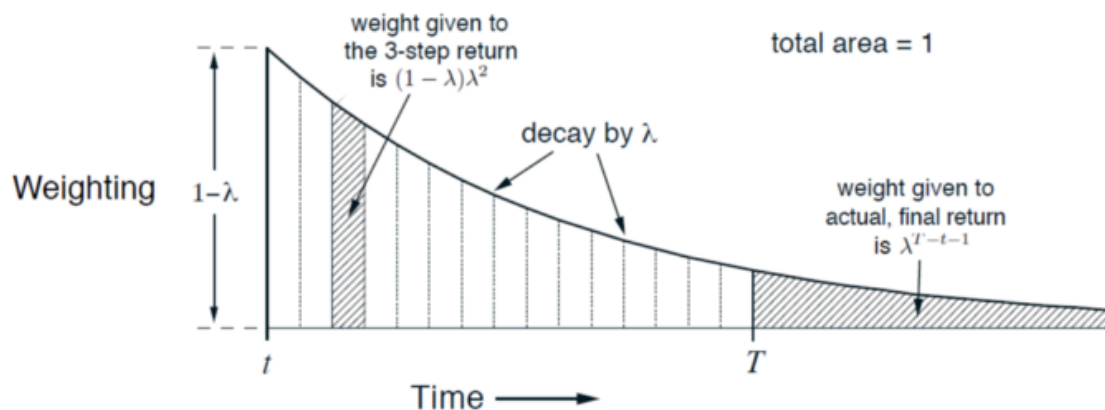
- The λ -return G_t^λ is also a biased sample of true value $v_\pi(s)$
- Can again apply supervised learning to “training data”:

$$\langle S_1, G_1^\lambda \rangle, \langle S_2, G_2^\lambda \rangle, \dots, \langle S_{T-1}, G_{T-1}^\lambda \rangle$$

- Forward view TD(λ)

$$\begin{aligned}\Delta \mathbf{w} &= \alpha(G_t^\lambda - \hat{v}(S_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}) \\ &= \alpha(G_t^\lambda - \hat{v}(S_t, \mathbf{w})) \mathbf{x}(S_t) \quad (\text{linear case})\end{aligned}$$

$$G_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} G_t^{(n)}$$



TD(λ) with Value Function Approx. : Forward view

- The λ -return G_t^λ is also a biased sample of true value $v_\pi(s)$
- Can again apply supervised learning to “training data”:

$$\langle S_1, G_1^\lambda \rangle, \langle S_2, G_2^\lambda \rangle, \dots, \langle S_{T-1}, G_{T-1}^\lambda \rangle$$

- Forward view TD(λ)

$$\begin{aligned} \Delta \mathbf{w} &= \alpha(G_t^\lambda - \hat{v}(S_t, \mathbf{w})) \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}) \\ &= \alpha(G_t^\lambda - \hat{v}(S_t, \mathbf{w})) \mathbf{x}(S_t) \quad (\text{linear case}) \end{aligned}$$


$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha \left[G_t^\lambda - \hat{v}(S_t, \mathbf{w}_t) \right] \nabla \hat{v}(S_t, \mathbf{w}_t), \quad t = 0, \dots, T-1$$

The λ - return gives an alternative way of moving smoothly between Monte Carlo and one-step TD methods that can be compared with the n-step bootstrapping way

TD(λ) with Value Function Approx. : Backward view

With function approximation, the *eligibility trace* is a vector $\mathbf{z}_t$ with the *same number of components as the weight vector* $\mathbf{w}_t$

TD(λ) with Value Function Approx. : Backward view

With function approximation, the *eligibility trace* is a vector $\mathbf{z}_t$ with the *same number of components as the weight vector* $\mathbf{w}_t$

In TD(λ), the eligibility trace vector is initialized to zero, is incremented on each time step by the value gradient, and then fades away by $\gamma\lambda$

$$\begin{aligned}\mathbf{z}_0 &\doteq \mathbf{0}, \\ \mathbf{z}_t &\doteq \gamma\lambda\mathbf{z}_{t-1} + \nabla\hat{v}(S_t, \mathbf{w}_t), \quad 0 \leq t \leq T,\end{aligned}$$

TD(λ) with Value Function Approx. : Backward view

With function approximation, the *eligibility trace* is a vector $\mathbf{z}_t$ with the *same number of components as the weight vector* $\mathbf{w}_t$

In TD(λ), the eligibility trace vector is initialized to zero, is incremented on each time step by the value gradient, and then fades away by $\gamma\lambda$

$$\begin{aligned}\mathbf{z}_0 &\doteq \mathbf{0}, \\ \mathbf{z}_t &\doteq \gamma\lambda\mathbf{z}_{t-1} + \nabla\hat{v}(S_t, \mathbf{w}_t), \quad 0 \leq t \leq T,\end{aligned}$$

Remarks:

- The eligibility trace keeps track of which components of the weight vector have contributed, positively or negatively, to recent state valuations, where recent is defined in terms of $\gamma\lambda$

TD(λ) with Value Function Approx. : Backward view

With function approximation, the *eligibility trace* is a vector $\mathbf{z}_t$ with the *same number of components as the weight vector* $\mathbf{w}_t$

In TD(λ), the eligibility trace vector is initialized to zero, is incremented on each time step by the value gradient, and then fades away by $\gamma\lambda$

$$\begin{aligned}\mathbf{z}_0 &\doteq \mathbf{0}, \\ \mathbf{z}_t &\doteq \gamma\lambda\mathbf{z}_{t-1} + \nabla\hat{v}(S_t, \mathbf{w}_t), \quad 0 \leq t \leq T,\end{aligned}$$

Remarks:

- The eligibility trace keeps track of which components of the weight vector have contributed, positively or negatively, to recent state valuations, where recent is defined in terms of $\gamma\lambda$
- The trace indicates the eligibility of each component of the weight vector for undergoing learning changes should a reinforcing event occur

TD(λ) with Value Function Approx. : Backward view

The reinforcing events we are concerned are the *one-step TD errors*

TD(λ) with Value Function Approx. : Backward view

The reinforcing events we are concerned are the *one-step TD errors*

The *TD error for state-value prediction* is

$$\delta_t \doteq R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t) - \hat{v}(S_t, \mathbf{w}_t)$$

TD(λ) with Value Function Approx. : Backward view

The reinforcing events we are concerned are the *one-step TD errors*

The *TD error for state-value prediction* is

$$\delta_t \doteq R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t) - \hat{v}(S_t, \mathbf{w}_t)$$

The weight vector is updated on each step proportional to the scalar TD error and the vector eligibility trace

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha \delta_t \mathbf{z}_t$$

TD(λ) with Value Function Approx. : Backward view

The reinforcing events we are concerned are the *one-step TD errors*

The *TD error for state-value prediction* is

$$\delta_t \doteq R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t) - \hat{v}(S_t, \mathbf{w}_t)$$

The weight vector is updated on each step proportional to the scalar TD error and the vector eligibility trace

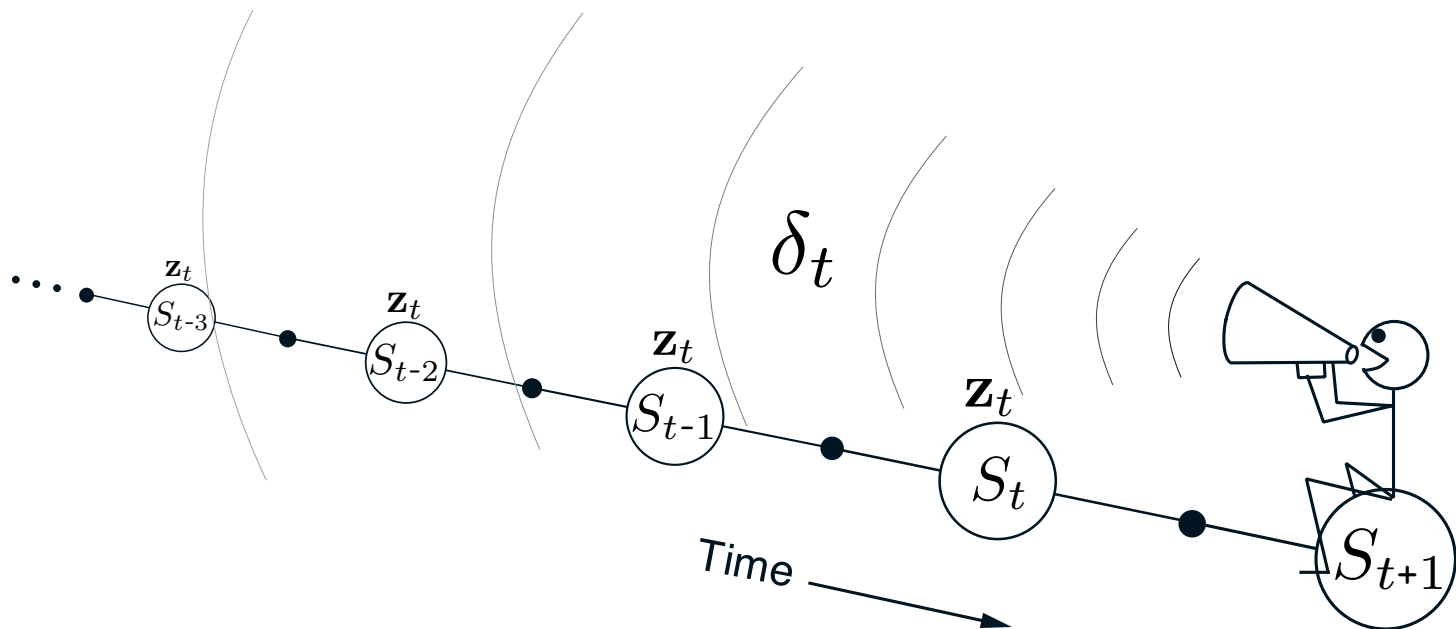
$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha \delta_t \mathbf{z}_t$$

When dealing with Value Function Approximation we do not update directly the states but the weight vector

TD(λ) with Value Function Approx. : Backward view

Remember that TD(λ) is oriented *backward* in time...

...at each moment we look at the current TD error and assign it backward to each prior state according to how much that state contributed to the current eligibility trace at that time...



TD(λ) with Value Function Approx. : Backward view

Semi-gradient TD(λ) for estimating $\hat{v} \approx v_\pi$

Input: π , $\hat{v} : \mathcal{S} \times \mathbb{R}^d \rightarrow \mathbb{R}$ ($\hat{v}(\text{terminal}, \cdot) = 0$), step-size α , decay-rate λ

Initialize value-function weights $\mathbf{w} \in \mathbb{R}^d$ arbitrarily (e.g. $\mathbf{w} = 0$)

Repeat (for each episode) :

Initialize S

$\mathbf{z} \leftarrow 0$

Repeat (for each step of the episode) :

Choose $A \sim \pi(\cdot | S)$

Take action A , observe R, S'

$\mathbf{z} \leftarrow \gamma \lambda \mathbf{z} + \nabla \hat{v}(S, \mathbf{w})$

$\delta \leftarrow R + \gamma \hat{v}(S', \mathbf{w}) - \hat{v}(S, \mathbf{w})$

$\mathbf{w} \leftarrow \mathbf{w} + \alpha \delta \mathbf{z}$

$S \leftarrow S'$

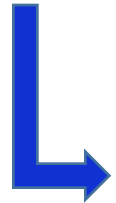
Until S' is terminal

TD(λ) with Value Function Approx. : Backward view

If $\lambda = 0$ then $\mathbf{z}_t \doteq \gamma \lambda \mathbf{z}_{t-1} + \nabla \hat{v}(S_t, \mathbf{w}_t)$

TD(λ) with Value Function Approx. : Backward view

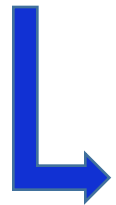
If $\lambda = 0$ then $\mathbf{z}_t \doteq \gamma \lambda \cancel{\mathbf{z}_{t-1}} + \nabla \hat{v}(S_t, \mathbf{w}_t)$


$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha (R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t) - \hat{v}(S_t, \mathbf{w}_t)) \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}_t)$$

We recover the 1-step return algorithm - **TD(0) algorithm!**

TD(λ) with Value Function Approx. : Backward view

If $\lambda = 0$ then $\mathbf{z}_t \doteq \gamma \lambda \cancel{\mathbf{z}_{t-1}} + \nabla \hat{v}(S_t, \mathbf{w}_t)$


$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha (R_{t+1} + \gamma \hat{v}(S_{t+1}, \mathbf{w}_t) - \hat{v}(S_t, \mathbf{w}_t)) \nabla_{\mathbf{w}} \hat{v}(S_t, \mathbf{w}_t)$$

We recover the 1-step return algorithm - **TD(0) algorithm!**

If $\lambda = 1$ then we have **Monte-Carlo approach** (*the equivalence can be proved in the off-line updates scenario*)

Forward and backward TD(λ) are equivalent in offi-line updates scenario

TD(λ) with Value Function Approx. : Backward view

Linear TD(λ) has been proved to converge in the on-policy case if the step-size parameter is reduced over time according to

$$\sum_{t=1}^{\infty} \alpha_t = \infty \quad \sum_{t=1}^{\infty} \alpha_t^2 < \infty$$

TD(λ) with Value Function Approx. : Backward view

Linear TD(λ) has been proved to converge in the on-policy case if the step-size parameter is reduced over time according to

$$\sum_{t=1}^{\infty} \alpha_t = \infty \quad \sum_{t=1}^{\infty} \alpha_t^2 < \infty$$

Convergence is not to the minimum-error weight vector, but to a nearby weight vector that depends on λ

TD(λ) with Value Function Approx. : Backward view

Linear TD(λ) has been proved to converge in the on-policy case if the step-size parameter is reduced over time according to

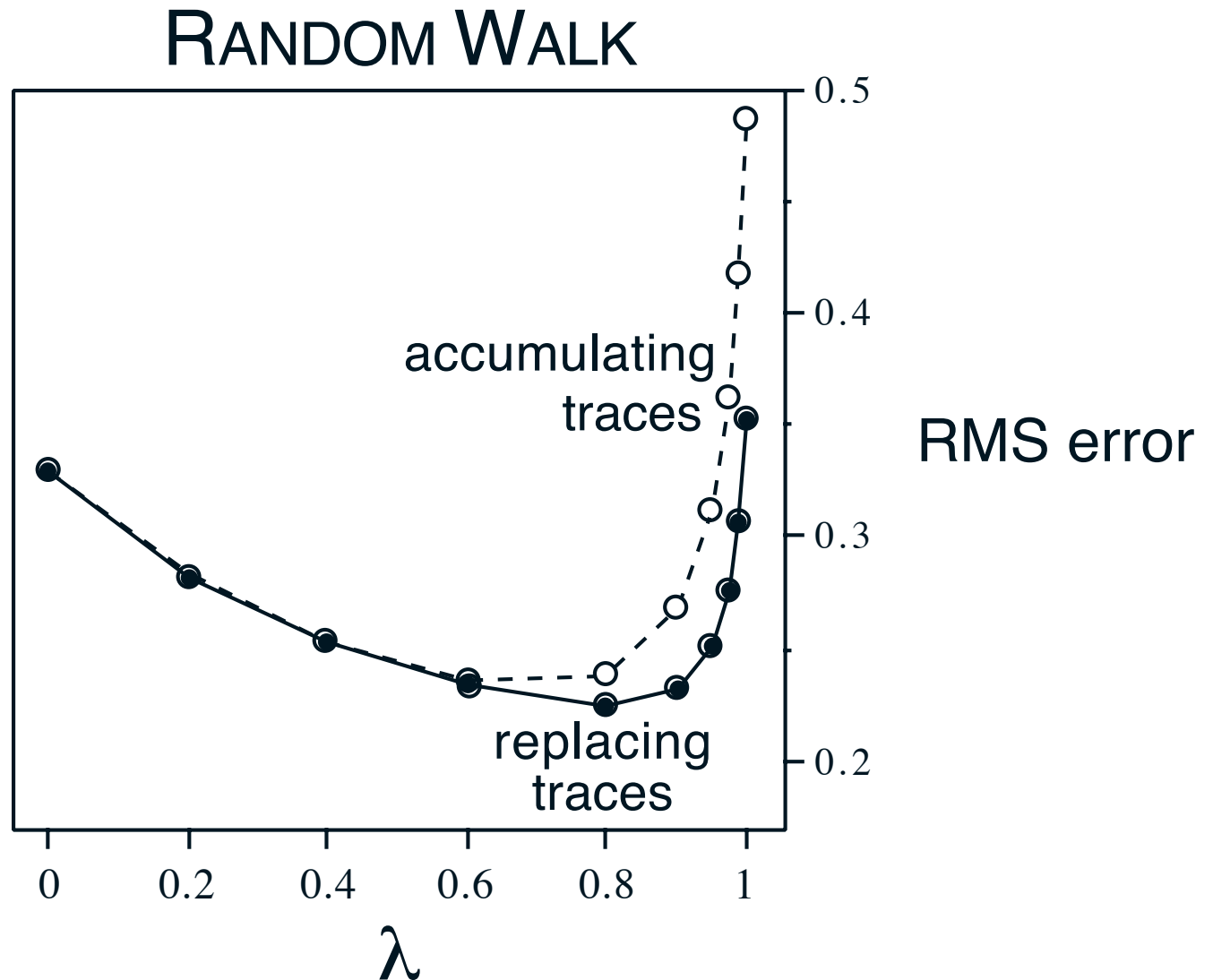
$$\sum_{t=1}^{\infty} \alpha_t = \infty \quad \sum_{t=1}^{\infty} \alpha_t^2 < \infty$$

Convergence is not to the minimum-error weight vector, but to a nearby weight vector that depends on λ

Bound for the continuing discounted case :

$$\overline{\text{VE}}(\mathbf{w}_{\infty}) \leq \frac{1 - \gamma\lambda}{1 - \gamma} \min_{\mathbf{w}} \overline{\text{VE}}(\mathbf{w})$$

Study of λ : Should we Bootstrap?

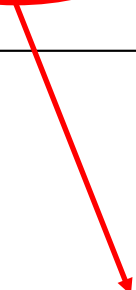


Convergence of Predictions Algorithms

On/Off-Policy	Algorithm	Table Lookup	Linear	Non-Linear
On-Policy	MC	✓	✓	✓
	TD(0)	✓	✓	✗
	TD(λ)	✓	✓	✗
Off-Policy	MC	✓	✓	✓
	TD(0)	✓	✗	✗
	TD(λ)	✓	✗	✗

Convergence of Predictions Algorithms

On/Off-Policy	Algorithm	Table Lookup	Linear	Non-Linear
On-Policy	MC	✓	✓	✓
	TD(0)	✓	✓	✗
	TD(λ)	✓	✓	✗
Off-Policy	MC	✓	✓	✓
	TD(0)	✓	✗	✗
	TD(λ)	✓	✗	✗



We will discuss some counterexamples next lecture

Material

The material illustrated today can be found in the first five paragraphs of Chapter 9 and in the first two paragraphs of Chapter 12.

In the book there are more descriptions and thoughts. For the final examination pay attention to what seen in the slides used for the lecture (as usual concentrate on the algorithms!)